

🗨 指针（Pointer）——给完全小白的讲解

本文适合：第一次接触指针、被 `\*` 和 `&` 吓到的人
阅读目标：看完后你至少能回答一句话——指针就是“地址的变量”

1. 一、先别急：指针到底是个啥？

1.1 🏠 生活中的比喻

想象一下：

- **变量** 就像一个 **房间**
- **变量的值** 就是房间里的人
- **内存地址** 就是这个房间的 **门牌号**

👉 **指针** 干的事是：

专门存“门牌号”的变量

所以一句话总结：

指针 = 用来存地址的变量

2. 二、普通变量 vs 指针变量

2.1 1 普通变量

```
1 | int a = 10;
```

Fence 1

含义是：

- 申请一块内存
- 里面放了数字 `10`

```
1 | 内存:
2 | [ 10 ] ← a
3 |
4 |
```

Fence 2

2.2 2 指针变量

```
1 | int *p;
```

Fence 3

含义是：

- `p` 是一个变量
- 它专门用来存 `int` 类型变量的地址

此时 `p` 还没指向任何房间，是“空导航”。

3. 三、`&` 和 `*` 是在干嘛？（重点！）

这是小白最容易懵的地方，我们一个一个来。

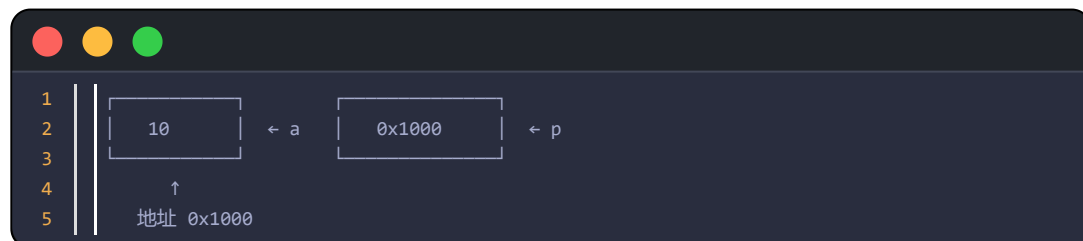
3.1 1 `&` —— 取地址

```
1 | int a = 10;  
2 | int *p = &a;
```

Fence 4

解释：

- `&a`：获取 `a` 的门牌号
- 把这个门牌号存进指针变量 `p`



Fence 5

类比：

- `a` 是房间
- `&a` 是门牌号
- `p` 是记着门牌号的小纸条

3.2 2 `*` —— 解引用（顺着地址找房间）

```
1 | printf("%d", *p);
```

Fence 6

意思是：

顺着 p 里面存的地址，找到那个变量，再取它的值

此时输出：

```
1 | | 10
```

Fence 7

👉 口诀：

*p = p 指向的那个变量

4. 四、用指针“遥控”变量（精髓）

```
1 | int a = 10;
2 | int *p = &a;
3 |
4 | *p = 20;
```

Fence 8

发生了什么？

- p 指向 a
- *p = 20 相当于：

```
1 | a = 20;
2 | 修改前: a = 10
3 | 修改后: a = 20
```

Fence 9

🎮 比喻：

- 指针就像 遥控器
- 不用碰电视（变量）
- 用遥控器（指针）就能改频道（值）

5. 五、为什么非要用指针？

5.1 1 让函数“真的改掉”变量

```
1 | void change(int *p)
2 | {
3 |     *p = 100;
4 | }
5 |
6 | int main()
7 | {
8 |     int a = 10;
9 |     change(&a);
10 |    printf("%d", a); // 100
```

✦ 如果不用指针：

- 函数只能改“副本”
- 用了指针，才能改“本体”

5.2 2 操作数组更方便

```
1 | int arr[3] = {1,2,3};
2 | int *p = arr; // 数组名就是首元素地址
3 | arr → [1][2][3]
4 |     ↑
5 |     p
```

6. 六、最容易犯的 3 个错误

6.1 ✖ 错误 1：野指针

```
1 | int *p;
2 | *p = 10; // ✖
```

原因：

- p 还没指向任何合法地址

✓ 正确做法：

```
1 | int a;
2 | int *p = &a;
```

6.2 ✖ 错误 2：把 `*p` 当成地址

```
1 | printf("%p", *p); // ✖
```

记住：

- `p` 才是地址
- `*p` 是值

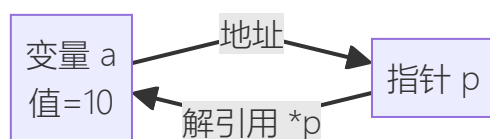
6.3 ❌ 错误 3：指针类型乱用

```
1 | int a = 10;  
2 | char *p = &a; // ❌
```

Fence 15

类型不匹配，后果不可控。

7. 七、一张图彻底记住



记忆公式：

```
p = &a  📌 记住 a 在哪  
*p  📌 拿到 a 本人
```

8. 八、结尾一句话总结

指针不是洪水猛兽，它只是“记住别人住哪儿”的变量。

📌 下一篇可以写：

- 指针和数组的关系（超重点）
- 二级指针（指针的指针）
- 指针在函数中的高级用法

如果你愿意，我可以按你课程进度继续给你写下一节。

☺ 指针一章 · 总结（终极版 · 一页吃透）

本总结用于期末复习 / 作业收尾 / 考前速记，只保留本章最核心内容。

1. 一、三句话概括整章

- 1 指针是变量，但它存的是“地址”，不是普通数据。
- 2 `&` 用来“取地址”，`*` 用来“通过地址访问变量”。
- 3 指针的本质作用：实现间接访问，从而修改原变量、操作数组。

2. 二、必须牢牢记住的等价关系

```
1 int a = 10;  
2 int *p = &a;
```

Fence 16

写法	含义	等价理解
a	变量本身	房间
&a	a 的地址	门牌号
p	存地址的变量	记门牌号的纸条
*p	p 指向的变量	进房间见到的人

Table 1

👉 核心等价式：

```
*(&a) = a  
*p 和 a 操作的是同一块内存
```

3. 三、指针最经典的三行代码（会这个就够了）

```
1 int a = 10;    // 定义变量  
2 int *p = &a;   // 指针保存地址  
3 *p = 20;       // 通过指针改变量
```

Fence 17

结论：

```
1 | a 的值变为 20
```

Fence 18

👉 指针不是自己改值，而是“代替别人改值”。

4. 四、为什么一定要学指针（考试/实用）

4.1 1 函数中修改原变量（必考）

```
1 | | change(&a);
```

Fence 19

- ✓ 用指针：改的是原变量
- ✗ 不用指针：只改副本

4.2 2 数组操作离不开指针

- 数组名 = 首元素地址
- 指针天然适合遍历数组

```
1 | | int *p = arr;
```

Fence 20

👉 数组 ≈ 指针 + 连续内存

4.3 3 后续知识的基础

- 二级指针
 - 字符串
 - 动态内存
- 👉 不懂指针，这些全部学不下去。

5. 五、本章 3 个致命易错点（重点）

5.1 ✗ 1. 野指针

```
1 | | int *p;  
2 | | *p = 10; // 错
```

Fence 21

- ✓ 必须先指向合法地址。

5.2 ✗ 2. 混淆 p 和 *p

表达式	含义
p	地址
*p	值

Table 2

5.3 ❌ 3. 指针类型不匹配

```
1 | | char *p = &a; // 错
```

Fence 22

👉 什么变量，就用什么指针类型。

6. 六、一张脑内结构图（考前默写）

```
1 | | 变量 a (值)
2 | |   ↑
3 | | &a |      *p
4 | |   |      ↓
5 | | 指针 p → a
```

Fence 23

口诀：

```
p = &a : 存地址
*p : 用变量
```

7. 七、本章最终结论（标准答案版）

指针是 C 语言中用于存储地址并实现间接访问的重要机制，
它使函数能够修改实参、数组能够高效处理，是理解内存模型的关键。

🔥 下一章推荐顺序：

- 1 指针与数组（高频考点）
- 2 字符指针与字符串
- 3 二级指针

☺ 指针与数组（整章讲解 · 小白友好版）

本章地位：高频考点 + 真正把指针“用起来”的一章
学完目标：看到数组、指针能在脑子里自动画图

1. 一、先给结论：数组和指针是什么关系？

数组名，本质上就是“指向数组第一个元素的指针”。

这一句话，值半章分数。

2. 二、从数组本身说起

```
1 | int arr[5] = {1, 2, 3, 4, 5};
```

Fence 24

你需要知道三件事：

- 1 数组在内存中是 **连续存放的**
- 2 每个元素大小相同（int 一般 4 字节）
- 3 `arr` 代表的是 **首元素地址**

```
1 | 内存示意：
2 |
3 | arr → | 1 | 2 | 3 | 4 | 5 |
4 |         ↑
5 |        arr[0]
```

Fence 25

3. 三、数组名 = 指针（但不完全一样）

3.1 1 数组名可以当指针用

```
1 | int *p = arr;
```

Fence 26

等价于：

```
1 | int *p = &arr[0];
```

Fence 27

此时：

○ `p` 指向 `arr[0]`

○ *p == arr[0]

3.2 2 但数组名不是普通指针

```
1 | arr = p; // ❌ 错误
2 | p = arr; // ✅ 正确
```

Fence 28

原因：

- 数组名是“常量地址”
- 指针是“变量地址”

👉 记忆：

数组名像“钉死的路标”，指针像“可改的导航”。

4. 四、指针访问数组的本质

```
1 | p[i] 等价于 *(p + i)
2 | arr[i] 等价于 *(arr + i)
```

Fence 29

👉 这不是语法技巧，是内存运算规则。

4.1 🏠 举个例子

```
1 | int arr[3] = {10, 20, 30};
2 | int *p = arr;
3 | 地址假设：
4 | arr      = 1000
5 | arr+1    = 1004
6 | arr+2    = 1008
```

Fence 30

表达式	结果
*p	10
*(p+1)	20
p[2]	30

Table 3

5. 五、指针遍历数组（核心用法）

5.1 1 下标方式

```
1 | for(int i = 0; i < 5; i++)
2 |     printf("%d ", arr[i]);
```

Fence 31

5.2 2 指针方式

```
1 | for(int i = 0; i < 5; i++)
2 |     printf("%d ", *(p + i));
```

Fence 32

5.3 3 指针自增（最像 C 语言）

```
1 | for(p = arr; p < arr + 5; p++)
2 |     printf("%d ", *p);
```

Fence 33

🎮 比喻：

指针像“手指”，一个一个点数组元素。

6. 六、数组作为函数参数（必考）

```
1 | void print(int *p, int n)
2 | {
3 |     for(int i = 0; i < n; i++)
4 |         printf("%d ", p[i]);
5 | }
```

Fence 34

调用：

```
1 | print(arr, 5);
```

Fence 35

👉 本质：

数组传参 = 传首地址

7. 七、二维数组与指针（了解层面）

```
1 | int a[2][3];
```

Fence 36

内存结构:

```
1 | | a[0][0] a[0][1] a[0][2] | a[1][0] a[1][1] a[1][2] |
```

Fence 37

常见写法:

```
1 | int (*p)[3] = a;
```

Fence 38

👉 读法:

p 是一个指向“包含 3 个 int 的数组”的指针

8. 八、本章 3 个高频错误

8.1 ❌ 1 越界访问

```
1 | arr[5] = 10; // ❌
```

Fence 39

👉 C 语言不检查越界，后果严重。

8.2 ❌ 2 把数组大小算错

```
1 | sizeof(arr) / sizeof(arr[0]) // 只在本函数有效
```

Fence 40

数组传进函数后:

```
1 | sizeof(p) // 得到的是指针大小
```

Fence 41

8.3 ❌ 3 混淆 arr、&arr、arr+1

表达式	含义
arr	首元素地址
&arr	整个数组地址

Table 4

☺ 字符数组 & 字符串（和指针强关联 · 超常考）

本章地位：期末必考 + 面试也常问
学完目标：彻底分清 字符数组、字符串、字符指针

1. 一、先给终极结论（不看后面也要记住）

字符串，本质上是“以 '\0' 结尾的字符数组”。

2. 二、字符数组是什么？

```
1 | | char a[5] = {'h','e','l','l','o'};
```

Fence 43

特点：

- 本质是数组
- 每个元素存一个字符
- 没有 '\0'，它不是字符串

```
1 | | 内存:  
2 | | | h | e | l | l | o |
```

Fence 44

👉 结论：

不是所有 char 数组都是字符串。

3. 三、字符串是什么？

```
1 | | char s[6] = "hello";
```

Fence 45

等价写法：

```
1 | | char s[6] = {'h','e','l','l','o','\0'};
```

Fence 46

内存结构：

```
1 | | h | e | l | l | o | \0 |
```

Fence 47

👉 关键点:

- `\0` 是字符串结束标志
- 没有它, 字符串函数会“迷路”

4. 四、`"hello"` 到底是什么? (常考)

```
1 | | "hello"
```

Fence 48

含义:

- 是一个 **字符串常量**
- 存在 **只读区**
- 结尾自动带 `\0`

```
1 | | "hello" → | h | e | l | l | o | \0 |
```

Fence 49

5. 五、字符数组 vs 字符指针 (必考对比)

5.1 **1** 字符数组

```
1 | | char s[] = "hello";
```

Fence 50

- 内容可修改
- 存在栈区

```
1 | | s[0] = 'H'; // ✓
```

Fence 51

5.2 **2** 字符指针

```
1 | | char *p = "hello";
```

Fence 52

- 指向字符串常量

内容不可修改

```
1 | | *p = 'H'; // ❌ 未定义行为
```

Fence 53

👉 超级结论（必背）：

项目	字符数组	字符指针
是否分配空间	是	否
是否可改内容	是	否
能否指向别处	否	是

Table 5

6. 六、字符串与指针的本质关系

```
1 | | char s[] = "abc";
2 | | char *p = s;
```

Fence 54

此时：

表达式	结果
s	首字符地址
p	首字符地址
*p	'a'
*(p+1)	'b'

Table 6

👉 所以：

字符串操作，本质就是指针移动。

7. 七、字符串输入（考试最爱坑）

7.1 ❌ 错误写法

```
1 | | char s[10];
2 | | scanf("%s", &s); // ❌
```

Fence 55

7.2 ✅ 正确写法

```
1 | | scanf("%s", s);
```

Fence 56

原因：

数组名本身就是地址

8. 八、常见字符串函数（必会）

位于 `string.h`

函数	作用	注意点
<code>strlen</code>	求长度	不算 <code>'\0'</code>
<code>strcpy</code>	复制	目标空间要够
<code>strcmp</code>	比较	比 ASCII
<code>strcat</code>	拼接	易越界

Table 7

9. 九、字符串作为函数参数

```
1 void print(char *p)
2 {
3     while(*p != '\0')
4     {
5         putchar(*p);
6         p++;
7     }
8 }
```

Fence 57

调用：

```
1 | print(s);
```

Fence 58

👉 本质：

传字符串 = 传首字符地址

10. 十、本章 3 个致命易错点

10.1 ❌ 1 忘记 `'\0'`

```
1 | char s[5] = "hello"; // ❌
```

Fence 59

10.2 ❌ 2 修改字符串常量

```
1 char *p = "hello";  
2 p[0] = 'H'; // ❌
```

Fence 60

10.3 ❌ 3 字符数组大小不够

```
1 char s[5];  
2 strcpy(s, "hello"); // ❌
```

Fence 61

11. 十一、一张字符串内存总图

```
1 字符串 s:  
2 | h | e | l | l | o |   
3 | ↑  
4 | p
```

Fence 62

口诀：

字符串 = char 数组 + '\0'
操作字符串 = 操作指针

12. 十二、本章一句话总结（标准答案）

字符串是以 '\0' 结尾的字符数组，
在使用中通常通过字符指针进行访问和传递，
字符数组与字符指针在存储方式和可修改性上存在本质区别。

🔥 下一章推荐：

- 二级指针
- 动态内存 malloc/free